LLM-Serving Scheduler Ranking Portability Across Workloads, Operating Regions, and Metrics

SOROUSH VAHIDI, New Jersey Institute of Technology, USA

Scheduler rankings are conditional on the workload, operating region, metric and execution model used to obtain them. We present a reproducible comparison methodology and an empirical study of a frozen, trace-driven LLM-serving simulator corpus: three released trace sources, 120 windows, six operating regions and 13 executed policies, including an 11-policy primary panel. The 9,360 configurations each have two deterministic verification passes, not independent stochastic repetitions. For the primary arrival-normalized weighted-goodput metric, cross-source Kendall correlations range from 0.551 to 1.000. Of 990 pairwise-policy comparison records, 36 satisfy the saved practical-margin and nominal interval criteria for a reversal; they span five policy pairs and all involve BurstGPT. A post-campaign cross-metric analysis yields a median correlation of 0.419 and deterministic top-choice agreement of 31.9%, while a post-hoc comparison finds overlap of maximizing policy sets in 57.8% of conditions. Independent numerical reproduction confirms the saved intervals, but their coverage and the historical FDR claim are not established. These characterize the historical simulator, not native serving rankings. Retrospective audits of the legacy GPU comparison and subsequent CPU conformance work leave faithful physical ranking transfer unverified. The results support reporting comparative rankings with their workload, metric, uncertainty and implementation boundaries.

CCS Concepts: • **Computing methodologies** → **Simulation evaluation**; • **Computer systems organization** → *Cloud computing*.

Additional Key Words and Phrases: LLM serving, scheduling, ranking portability, trace-driven simulation, performance evaluation

1 Introduction

A scheduler ranking is a conditional statement. It compares a fixed set of implementations under a workload, an operating region, an objective and an execution model. A ranking obtained from one trace cannot automatically be used to select a policy for another trace; nor does agreement on average throughput imply agreement on deadline attainment or tail latency. This paper studies how such comparative conclusions transfer, rather than introducing a new scheduler.

LLM serving makes this question concrete. Continuous batching and paged key–value (KV) memory management permit concurrent requests [11, 24]. Chunked prefill changes competition between prompt processing and decoding [2]; disaggregation separates their resource requirements [15, 27]. Production traces differ in arrival bursts, prompt lengths and output lengths [18, 21]. These established observations motivate testing the stability of a comparison across conditions. They do not establish which policies will reverse order, or even that reversals must occur in a particular evaluation.

We study the LLM-Serving Scheduler Portability Benchmark (LSSP), a frozen, trace-driven simulator campaign. Its primary corpus contains 120 windows from three separately released trace sources. A 13-policy execution panel, including 11 primary policies and two style approximations, is evaluated in six FIFO-referenced operating regions. There are 9,360 distinct configurations and two deterministic verification passes per configuration. The passes check repeatability of the simulator; they are not independent samples. Historical resampling variation is instead assessed across workload windows, subject to the dependence limitations in Section 10.

The contribution is a reproducible evaluation methodology and an empirical characterization of *simulated* scheduler-ranking portability. The methodology combines a fixed comparison panel, matched policy outcomes, explicit metric

Author's Contact Information: Soroush Vahidi, sv96@njit.edu, New Jersey Institute of Technology, Newark, New Jersey, USA.

2026. Manuscript submitted to ACM

eligibility, rank agreement, practical-margin reversal classification and a finite-corpus ranking-recovery analysis. Kendall correlation, bootstrap intervals and false-discovery correction are standard statistical tools, not a new algorithm or theoretical result. The scientific object is the reliability and scope of the comparative conclusion produced by their application.

1.1 Why comparative validity is the evaluation target

A performance evaluator often has two decisions to make: which implementation to compare, and which evidence would justify carrying its ordering into a new setting. The second decision can fail even when the first experiment is reproducible. An average over a trace can hide a policy pair whose ordering changes elsewhere. Conversely, a small rank change can be irrelevant if both implementations attain almost identical utility. We therefore separate agreement of the full panel, individual pair reversals, and changes in the set of preferred policies. These are complementary questions with different denominators.

This distinction matters for modeling as well as for deployment. A simulator can be repeatable while representing service demands or resource constraints differently for different policies. Its ranking is then a property of the complete policy–model combination. Reporting only a leaderboard would encourage readers to attribute all differences to the scheduling discipline. Reporting the model boundary, absolute outcome levels and comparison support makes that attribution testable. This paper uses the historical corpus to expose where the evidence ends rather than treating a policy's implementation name as validation.

The unit of observation is equally consequential. Thousands of execution records need not provide thousands of independent observations: policies reuse the same windows, regions are transformations of those windows, and the second deterministic pass supplies no sampling variation. Similarly, more metric pairs do not create new workload coverage. A portable comparison needs a clear account of what was resampled and what remained fixed. This motivates preserving the full comparison grid, including stable, tied and undefined cases, rather than reporting only reversals.

1.2 Contribution and evidential scope

The primary ANWG ranking correlations range from 0.551 to 1.000 across 18 source-pair/region comparisons. Of 990 pairwise-policy comparison records, 36 meet the saved practical-margin and nominal interval reversal criteria; these span five policy pairs and all involve BurstGPT. Most comparisons therefore retain their order. A separately labeled post-campaign analysis shows a median cross-metric Kendall correlation of 0.419 and agreement of the deterministic top-choice rule in 316 of 990 conditions. A retrospective audit additionally distinguishes deterministic choice from the existence of a common maximizing policy, and saved interval support from calibrated inference. Agreement and disagreement are both findings; neither is assumed in advance.

The resulting contribution has three parts: a fixed evidence organization that separates ranking agreement from practical pairwise change; a fully identified simulator case study spanning source, operating region and objective; and an independently reproducible audit showing how ties, missingness and inferential assumptions alter the meaning of the saved summaries. The audit is post hoc. It strengthens traceability, not the prospective status of the experiment. No new scheduler, rank statistic, coverage theorem or learned transfer predictor is proposed.

These are observations of historical simulator implementations. Later source and engineering audits identified limits in partial-prefill execution, native admission, KV allocation, preemption and execution acknowledgment. The historical GPU comparison also has metadata and replay limitations. It cannot establish faithful native-policy ranking transfer. Accordingly, this paper neither claims physical validation of its simulated reversals nor generalizes them to GPU

architectures or parallel deployments. The distinction is central to the domain of validity, not a qualification added after interpreting the numerical results.

The four retrospective manuscript questions and their mapping to all six original protocol questions appear in Section 3. Sections 4.2 and 5 define the model and experimental design; Sections 6, 7 and 8 separate operating-region behavior, cross-source portability and cross-metric portability. Robustness and practical implications precede an explicit account of threats to validity.

2 Background and Related Work

2.1 Serving mechanisms and workload variation

Prefill processes an input context; autoregressive decoding generates new tokens while retaining KV state. Iteration-level scheduling in Orca [24], PagedAttention in vLLM [11] and stall-free chunked prefill in Sarathi-Serve [2] address different batching and memory constraints. DistServe and Splitwise separate prefill and decode resources [15, 27]; Llumnix rebalances instances [19]; Mooncake organizes serving around KV state [16]. Fairness-aware token accounting [17], service-length-aware packing [10], learned ordering [9] and SLO-aware scheduling [4] illustrate why a single scalar ranking need not preserve every deployment objective. We compare historical simulator policies inspired by such mechanisms, not these systems as interchangeable production implementations.

BurstGPT, Azure and Bailian expose different production-derived request populations [3, 12, 21]. Characterization and generation work examines workload heterogeneity, including KV reuse and changes over time [13, 20, 22]. BurstGPT and Azure are separately released sources associated with Azure infrastructure; their comparison does not isolate a provider effect. Observed prompt lengths can also have different conversation/context semantics. LSSP retains these distinctions when interpreting portability.

2.2 Simulation and evaluation methodology

Vidur and LLMServingSim provide infrastructure for evaluating serving configurations [1, 6, 7]. LSSP does not claim a more accurate simulator. Its emphasis is the comparative conclusion induced by a fixed model, together with the restrictions on transporting that conclusion beyond the model.

Workload-dependent performance is established background. Work on serving workload selection [14], production-trace dispatching [23], and workload resampling in parallel job scheduling [26] provides close methodological precedents. The distinctive scope pursued here is a common, frozen panel with jointly reported source, region and metric comparisons, explicit reversal denominators, and uncertainty conditional on the available windows. It is not a priority claim that workload sensitivity or ranking comparison was previously unknown.

Kendall's tau-b measures ordinal concordance while accommodating ties; Spearman correlation and top-k overlap provide complementary summaries. Neither a positive aggregate correlation nor a shared top choice rules out pair-specific reversals. Practical margins distinguish a numerically changed order from an operationally meaningful difference under a stated convention. These standard measures serve a reproducible comparison protocol. Our finite-corpus recovery study estimates stability relative to the frozen reference ranking, not a theorem about population sample complexity.

2.3 What is added beyond standard ranking statistics

Workload resampling has a direct performance-evaluation precedent. Zakay and Feitelson's conference study decomposes traces into user subtraces and recombines them while preserving activity cycles [25]. The shared concern is that the structure of the workload affects the reliability of a scheduler comparison. LSSP resamples already selected request windows instead; it does not implement user-level trace reconstruction or establish that its windows are independent. The distinction prevents borrowing a validity argument from a resampling design with different observational units.

Demšar's comparison framework uses ranks across multiple datasets to compare algorithms, including Friedman omnibus tests and subsequent comparisons [8]. LSSP applies standard rank tools to fixed policy implementations, but asks an additional descriptive question: whether their order is preserved when the condition or metric changes. A pooled difference between policies is not itself evidence of such a change. A method can outperform another in every source and still produce a strong omnibus statistic. The source-pair reversal table is therefore a distinct object, not an automatic interpretation of Friedman significance.

The methodological addition is the explicit linkage between comparison objects: each ranking carries its implementation panel, observed-window support, utility definition, practical threshold and tie rule. The post-hoc audit shows that these choices have measurable consequences in one corpus. For example, agreement of deterministic top choices and intersection of maximizing sets yield different answers without any change to the outcomes. This is evidence for reporting both objects when ties are common; it is not a claim that a new estimator has been invented.

The study also separates three validation tasks that can otherwise be conflated: reproducing an analysis from frozen numbers, assessing its sampling assumptions, and checking whether the simulator represents the intended serving mechanisms. Exact numerical reproduction addresses only the first. The inferential audit addresses limits of the second; the legacy native-runner audit exposes unresolved parts of the third. This separation is the paper's relevance to modeling and performance evaluation, but it also limits the strength of its systems conclusions. A benchmark with correctly implemented standard statistics still needs a defensible comparison model and sufficiently representative inputs.

3 Research Questions and Portability Measures

We organize the manuscript around four questions that the frozen outcomes can address. This is a retrospective presentation change, not a new preregistration. "Answer" below means a conditional description of the specified implementations and corpus, not a population or native-system claim.

Q1: Source and region portability. How much do aggregate policy orderings agree across the three released sources and six operating regions?

Q2: Practical pairwise reversals. Which policy pairs change order with more than the frozen 10% relative margin on both sides, and how sensitive is their saved interval support to the analysis assumptions?

Q3: Objective sensitivity. How do metric choice, tied optima, observation support and the saved alternative SLO definitions change preferred-policy conclusions?

Q4: Temporal and finite-reference stability. How stable are rankings across the available temporal splits and subsets of the 40-window source reference?

Q3 uses post-campaign cross-metric and SLO extension artifacts. Broad metric and SLO sensitivity were anticipated in the historical plans; that does not make the eventual extension implementation, its selected region coverage, or the present matched-support and set-valued analyses prospectively frozen primary results. The original questions and plans remain unchanged in the artifact.

Table 1. Mapping to all six original protocol questions. The four manuscript questions above are retrospective; extension results remain post-campaign.

Original protocol question	Evidence status and manuscript location
RQ1: cross-source stability	Answered descriptively for this corpus (Q1, Q2); source release does not establish sampling independence.
RQ2: temporal/provider/domain shifts	Partially answered (Q1, Q4). Temporal splits exist; provider and domain effects are not isolated.
RQ3: synthetic-to-real transfer	Unanswered at the intended fidelity; pilot context only.
RQ4: observable reversal characteristics	Partially answered descriptively; sparse association fits do not identify causes (Section 9.4).
RQ5: independent windows needed for stability	Partially answered (Q4): finite-reference recovery, not population sample complexity.
RQ6: faithful native ranking transfer	Unanswered; legacy runner defects prevent that interpretation (Section 10.2).

3.1 Comparison object and statistical unit

Let $R(W, M, L)$ denote the ordering of a fixed policy panel under source or window set W, metric M, and operating region L. Collapse the two identical deterministic passes before aggregation. Policies share windows within a source/region; different sources are not paired request by request. A whole window is the historical resampling unit. This preserves within-window policy dependence but assumes exchangeability between windows; non-overlap alone does not establish that assumption.

For defined outcomes $y_{w,p,m}$, the historical aggregate is

$$\bar{y}_{p,m} = |\mathcal{W}_{p,m}|^{-1} \sum_{w \in \mathcal{W}_{p,m}} y_{w,p,m}, \tag{1}$$

where $\mathcal{W}_{p,m}$ contains the observed windows for policy p and metric m. Empty support gives no aggregate. Pairwise difference intervals instead use $\mathcal{W}_{a,m} \cap \mathcal{W}_{b,m}$. These estimands coincide for complete ANWG observations, but need not coincide for conditional metrics. Section 8 examines this distinction without imputing unobserved outcomes.

3.2 Metrics

The primary metric, Arrival-Normalized Weighted Goodput (ANWG), is

$$\text{ANWG} = \frac{\sum_{i \in S} w_i}{\sum_{i \in R} w_i}, \tag{2}$$

where R contains all arriving requests, $w_i \geq 1$ is priority weight, and S contains requests completed at or before their deadline. Dropped, unfinished and late requests retain denominator weight but receive no numerator credit. Every window has arrivals. ANWG, completion fraction and weighted completion fraction are therefore always defined.

The other eligible metrics are SLO-violation rate, weighted goodput, mean and p95 latency, request and token throughput, and mean and p95 TTFT. The first six require a completion; TTFT additionally requires a recorded first-token time for a completed request. The executed ranking schema does not contain p99 latency or TPOT. Undefined observations are omitted, never assigned a zero, loss or tie. A condition can remain estimable even when its policy means use different window subsets. Lower-is-better metrics are negated for cross-metric comparisons.

Kendall tau-b preserves equal values as ties. With concordant and discordant pair counts C, D and one-sided tie counts T_x, T_y,

$$\tau_b = \frac{C - D}{\sqrt{(C + D + T_x)(C + D + T_y)}}. \tag{3}$$

A zero denominator is undefined. Top-k and exact-order recovery instead sort by metric value and then policy ID. Thus a single top choice need not be a unique optimum. Q3 reports both the historical choice and a post-hoc maximizing-set comparison.

3.3 Saved uncertainty and reversal taxonomy

The historical procedure draws 2,000 samples of windows with replacement and records the 2.5th and 97.5th percentiles. We call these *nominal 95% resampling intervals*; their reproduction does not establish 95% coverage under temporal dependence or for discontinuous, tied rankings. Undefined rank statistics are discarded before percentile calculation. This occurs in 30 of 198 source comparisons; the two primary LOW comparisons involving BurstGPT discard 10 and 12 of 2,000 draws. The resulting intervals condition on an estimable resampled ranking, even though all primary ANWG cell values are observed. Cross-source ranking samples are drawn separately. Cross-metric ranking samples share window indices. Cross-metric pairwise difference intervals use separate seeded marginal draws for each metric, a distinct procedure from the paired ranking bootstrap.

For policy pair a, b, the saved classification first compares separately aggregated differences $d_x = \bar{y}_{a,x} - \bar{y}_{b,x}$ and d_y. Opposite nonzero signs are required. Each absolute difference must exceed 10% of the absolute value of the losing policy in that condition. A zero loser makes this margin unestimable, except that an exact tie has zero difference and margin. The threshold is a historical operational convention, not an established deployment-utility cutoff or a confidence bound on the effect size.

The five exclusive labels are: undefined/unestimable; stable (including exact ties); microscopic sign change (at least one margin at most 10%); practical sign change with a saved interval including zero; and `SUPPORTED_PRACTICAL_REVERSAL` (both margins pass and both saved difference intervals exclude zero). We retain the final label only as an artifact category, abbreviated *saved-supported*. It does not assert calibrated statistical significance. The sequential rule skips interval calculation for stable and microscopic records; absence of the wide-interval class does not establish equivalence between practical and statistical criteria.

The saved tail quantity is

$$p_s = \min\{1, 2\min[\widehat{P}(d_s^* \le 0), \widehat{P}(d_s^* \ge 0)]\}, \qquad p_{\text{pair}} = \max(p_x, p_y). \tag{4}$$

This is an empirical bootstrap tail fraction, not an exact binomial sign test. The maximum has an intersection–union interpretation only if its component tests are valid for the intended nulls. Here null calibration, data-dependent directional screening and window dependence are not established. Nor do zero-excluding intervals test the stronger hypothesis that both population margins exceed 10%.

The historical BH calculation uses only records reaching the interval stage: families are metric/region for cross-source reversals and source/region/metric-pair for cross-metric disagreements. Independence or the required positive dependence of valid null p-values is not shown [5]. Consequently, the saved flags are reproduced bookkeeping results, *not a verified 5% FDR guarantee*, even within families. Section 9.5 reports bounded alternative bookkeeping and dependence sensitivities; none repairs null calibration by changing a denominator alone.

Table 2. Units in the frozen campaign. Comparison counts describe coverage, not independent replications.

Quantity	Interpretation
120 windows	40 per source, 200 requests per window
9,360 configurations	120 windows × six regions × 13 policies
18,720 executions	Two deterministic passes per configuration
18 primary rank comparisons	Three source pairs × six regions
990 primary reversal records	18 conditions × 55 policy pairs
990 cross-metric conditions	Three sources × six regions × 55 metric pairs
54,450 metric disagreements	990 conditions × 55 policy pairs
500 draws per recovery budget	Subsets of the fixed reference corpus

3.4 Evidence denominators

All reported fractions condition on these enumerated comparison sets. Increasing the metric or policy panel creates more records without adding workload evidence. The separate 240 legacy physical executions do not enlarge the simulator's inferential sample.

4 Evaluation Model and Assumptions

4.1 Scheduler Panel

The PRIMARY panel comprises 11 policies, executed alongside 2 additional `STYLE_APPROXIMATION` policies used only for a mandatory robustness check (13 policies executed in total): a classical arrival-order baseline (`fifo`, also the load-calibration reference policy), a deadline-aware policy (`edf`), a slack/urgency-aware policy (`least_laxity_first`), a service-length-aware policy (`estimated_service_time_first`), a fairness-aware policy (`weighted_fair_share`), a KV/memory-pressure-aware policy (`kv_constrained_online`), an admission-control/SLO-guard policy (`admission_control`), and four historically named simulator implementations of external mechanisms: vLLM-style token-budget continuous batching in both its pre-chunked-prefill and chunked-prefill forms (`vllm_faithful`, `vllm_chunked_prefill_faithful`), Sarathi-style chunked prefill (`sarathi_faithful`), and an SLO-aware scheduling policy in the SLAI/RAD family (`slai_faithful`). The two `STYLE_APPROXIMATION` robustness policies are a token-budget-proxy style approximation (`vllm_style_token_budget`) and an SLO-guard-proxy style approximation (`scorpio_style_slo_guard`); two further implementations requiring disaggregated/cross-instance simulator semantics (`distserve_faithful`, `llumnix_faithful`) are designated for a secondary stratum, never pooled with the single-GPU colocated PRIMARY leaderboard, and one candidate (`apt_serve_faithful`) is excluded entirely as `scaffolding_only` – not a validated reimplementation at the time of this freeze.

A policy enters the PRIMARY panel only if, at the time it was added to the comparability audit – frozen a full day before the discriminability pilot (§5.8) executed – it satisfies an outcome-independent selection rule: its mechanism predates this protocol's freeze, it occupies a mechanism family not already saturated by an existing panel member, it was judged to have adequate implementation fidelity and passed its own unit tests, it is simulator-semantically compatible with the single-GPU colocated execution model, and it is independently reproducible from a cited reference. No policy was added to or removed from the panel in response to the pilot's outcome, and mechanism families it identified as under-implemented (preemptive/multilevel scheduling in the FastServe sense, JITServe, LMetric) were deliberately left as documented future-work candidates rather than fast-tracked into the panel after seeing the pilot's BurstGPT collapse.

4.2 Execution Model

Each (window, policy, operating-region) cell is executed as a deterministic discrete-event simulation. The entities are requests, one or more simulated GPU resources, a waiting queue, and policy-controlled active batches. Each request carries an arrival time, prompt-token count, target output-token count, priority weight, and, where required by a policy or metric, an SLO/deadline value. Simulator state includes the pending-arrival list, waiting queue, active requests per GPU, request phase and progress (prefill remaining, decoded tokens, first-token time), current KV-token footprint, per-step active-batch and utilization histories, completed and dropped requests, and observational mechanism telemetry (§4.4).

Time advances in fixed serving steps. At each step, all requests with arrival time less than or equal to the current simulation time are enqueued; the simulator exposes an observable state to the scheduling policy; the policy returns an action specifying admissions and, for policies that support them, preemption, swapping, migration, held decodes, or prefill-control decisions; the simulator validates the action against the resource model and applies it; active requests then advance by one service step; completed requests are removed and recorded; and per-step utilization, batch, KV, and contention histories are updated. The primary benchmark uses the colocated single-GPU-compatible stratum of this model; disaggregated or cross-instance semantics exist for secondary policies but are not pooled with the PRIMARY leaderboard (§4.1).

The resource model includes active-sequence capacity, batch/token-budget constraints where a faithful policy uses them, and KV-memory accounting. KV occupancy is tracked as demand relative to the configured nominal `max_kv_tokens`; only KV-aware policies enforce this limit at admission, so occupancy above 1.0 is interpreted as demand exceeding nominal capacity rather than as an instrumentation failure. In the default backward-compatible mode, prefill is instantaneous and each active decoding request produces one output token per step. In the prefill-aware mode used by faithful serving-style policies, requests may first consume prefill budget before decoding begins; decoding requests produce one token per step; chunked-prefill policies may limit the prefill work admitted into a step; and prefill/decode contention and token-budget saturation are recorded as diagnostics. Admission-control and SLO-aware policies may decline otherwise waiting requests or hold decode work according to their policy semantics; SLO/deadline outcomes are then evaluated through the metric contract (§3.2).

Simulation terminates when all arrivals have been processed and no request remains waiting, active, migrating, or relocating, or when the configured post-arrival drain limit is reached. Requests left in the waiting or transfer states at termination are treated as dropped. Metrics are computed from completed requests, dropped requests, the full arrival population, and the per-step histories; no completion-conditioned metric is imputed when no request completes (§3.2). Given identical requests, configuration, policy, and operating region, the simulator consumes no randomness and produces bit-identical outputs; this was verified by a repeated-run determinism test. This determinism is what permits the paired design of §3.1: two policies' outcomes share the same input window without independent simulation noise. Differences also include policy-dependent service modes, resource enforcement and information access; they cannot be attributed to scheduling order alone.

4.3 Historical fidelity labels and execution limits

The frozen registry distinguishes repository-native classical policies, simulator proxies, style approximations and a class named `FAITHFUL_EXTERNAL`. We retain identifiers such as `slai_faithful` solely as immutable data keys. These names and their historical unit-test gates do not certify native execution equivalence. Policy-specific shadow progress

Table 3. Main simulation-model scope and assumptions used in the PRIMARY ranking-portability study.

Model component	Treatment in this study
Arrivals	Trace-driven request arrivals from frozen 200-request windows.
Service progression	Fixed-step serving model; decode emits one token per active decoding request per step.
Prefill/decode	Instant prefill in legacy mode; explicit prefill budget and contention diagnostics in prefill-aware modes.
Resource limits	Active-sequence, token-budget, and KV-demand constraints as implemented by the configured policy/model.
Scheduling decisions	Policies observe simulator state and decide admission, ordering, and supported control actions.
Metrics	Computed from arrivals, completions, drops, deadlines, and per-step histories; undefined conditional metrics are excluded, not imputed.
Omitted hardware effects	No claim of absolute latency or throughput fidelity to physical GPUs; kernel, networking, allocator, and hardware-noise effects are abstracted in the numerical campaign.

and memory state need not equal the shared execution state. Per-request token intentions are not always carried as exact native runner plans; the resource model is not a native KV allocator.

Source-verified follow-up work found different final-prefill sampling, admission and recomputation semantics in pinned vLLM 0.27.1. Native scheduling can advance optimistic progress before runner acknowledgment; recomputation retains previously emitted output for replay. Historical simulator eviction can instead reset output and first-token progress. These differences are plausible sources of comparative bias, but no measured ranking correction or reversal direction follows from discovering them. We neither repair historical outputs nor relabel them as v2 results.

The numerical campaign remains useful as a controlled comparison of the specified implementations. Determinism verifies repeat execution within that model. It does not establish absolute hardware timing accuracy, mechanism equivalence to production systems, or physical portability of relative rankings.

4.4 Mechanism Telemetry

Every cell records seven groups of simulator telemetry: queue depth; active-batch saturation; prefill/decode contention; KV occupancy; admission activations; preemption/reorder events; and token-budget saturation. Means and maxima supplement several groups. A zero can mean that an operation did not occur or that the configured model does not represent that mechanism. These are simulator observations, not GPU counters. The descriptor association analysis in Section 9.4 does not establish a causal explanation of the observed reversals.

4.5 Implementation identity and comparison scope

A policy label is not a complete treatment definition. The shared simulator service model, admission constraints, policy-local shadow state and input information together determine the observed outcome. When a policy uses request-length information that is available in a trace-driven replay, that information is part of the historical experiment; it must not be represented as an online estimate validated on a production engine. Comparability therefore means controlled replay of the frozen inputs and implementations, not identical internal information or resource enforcement across every named mechanism.

The execution schema records configuration, source, window, policy and repetition identities alongside simulator and manifest hashes. Reproducing a table requires those identities as well as its numerical values. A future corrected simulator or common-native adaptation must receive a separate version and evidence namespace. It cannot replace

Table 4. Frozen panel interpretation. Registry labels are historical identifiers; none of these rows certifies native serving equivalence.

Policy or group	Historical comparison role
`fifo`	Arrival-order baseline and FIFO calibration reference
`edf`, `least_laxity_first`	Deadline and slack/urgency ordering
`estimated_service_time_first`	Service-length-aware ordering under the simulator's information model
`weighted_fair_share`	Priority-weighted fairness discipline
`kv_constrained_online`	Simulator KV-pressure proxy
`admission_control`	Admission/SLO guard
`vllm_faithful`	Historical continuous-batching implementation
`vllm_chunked_prefill_faithful`	Historical chunked-prefill implementation
`sarathi_faithful`	Historical Sarathi-inspired implementation
`slai_faithful`	Historical SLAI-inspired implementation
`vllm_style_token_budget`, `scorpio_style_slo_guard`	Two style approximations outside the primary ranking panel

a policy's old implementation while retaining the old results as if they described the new semantics. That version boundary permits an honest historical comparison without freezing known limitations into future serving work.

5 Workloads and Experimental Design

5.1 Sources

LSSP's primary benchmark corpus draws from three independently released workload sources: BurstGPT [21], Microsoft's official Azure 2024 LLM-inference trace [12, 18], and Alibaba Cloud's Bailian/Qwen anonymized usage-trace release [3]. Azure's 2023 conversation/code traces [15] and windows from an earlier discriminability pilot (§5.8) contribute additional context but are not part of the primary 120-window matrix beyond the 10-per-source pilot windows reused as described below. TraceLab [28] is retained as an out-of-distribution, coding-agent workload family for future expansion, not as one of the three primary sources, because its independence from a prior 512-window public sweep cannot be fully verified from public artifacts alone.

BurstGPT's own reported collection methodology logs traffic from a regional customer of Microsoft's Azure OpenAI GPT service, and this project's Azure trace is Microsoft's own official release of internal Azure LLM-inference-service traffic. Both therefore sit on the same underlying cloud provider's infrastructure. We describe BurstGPT and Azure as distinct, independently released workload sources – different collection methodology, different customer population, different release provenance and version history – and we do not describe them as independent providers. Bailian/Qwen (Alibaba Cloud) is a genuinely different underlying platform from both.

5.2 Representation and Provenance

Each source is ingested through a dedicated, field-level-audited adapter that maps the source's native schema onto a common internal request representation (arrival time, input tokens, output tokens, total tokens, session identifier where available, model class). Native-field provenance is separate from benchmark synthesis: absent tenant, KV-reuse and failure fields are not inferred, whereas the benchmark supplies priority/SLO parameters for policies and metrics where necessary. These are synthetic experimental attributes, not source-observed service guarantees. License status for each source was verified before any confirmatory use of a real, full-scale sample (for example, BurstGPT is under CC-BY-4.0).

5.3 Window Construction

The benchmark corpus freezes exactly 120 windows of 200 requests each: 40 per source (`azure_llm_2024`, `bailian_-qwen`, `burstgpt`), each composed of 10 windows reused verbatim from the discriminability pilot (§5.8) plus 30 new windows drawn by a deterministic, outcome-blind, source-symmetric pinned-extension sampler that uses only valid-row index ranges and fixed seeds – never scheduler metrics, policy identities, or the pilot's tie/non-tie outcomes (verified by structural audit to contain no BurstGPT-outcome-targeting signal). Windows are assigned to EARLY/MIDDLE/LATE chronology strata by deterministic relative chronology (60/30/30 windows respectively); the reused pilot windows are all EARLY. The freeze was independently validated – unique window identities, exact per-source and reused/new counts, fixed `request_count = 200`, no prohibited overlap, valid chronology, and absence of any scheduler/policy/load-calibration outcome field in the selection rule – and recorded with a canonical content hash (excluding generation timestamp; full values and verification records are in the preserved artifact hash ledger) before any load calibration or scheduler outcome was generated.

5.4 Descriptors

Each window is described by a 28-feature descriptor vector spanning prompt- and output-token-length statistics (mean, p90, long-prompt fraction at the 512/2048/8192-token thresholds), arrival-timing statistics (mean arrival rate, inter-arrival percentiles), and derived load-pressure proxies. Three deterministically redundant pressure-proxy features ($R^2 = 1.0$ against other included features) are retained in the underlying dataset but excluded from the classifier input used in §5.5. Prompt-token figures compose differently across sources for architectural reasons: TraceLab's and Azure's prompt-token fields are potentially multi-turn-cumulative context, while BurstGPT's is a single per-request figure. This is a property of how each source's underlying workload is architected (a growing agent/conversation session versus a single stateless API call), not an adapter artifact, and it is preserved as an explicit caveat on every prompt-length-based claim below.

5.5 Characterization scope

A separate descriptor study used 400 windows across four classes, including TraceLab. It is not part of the 120-window outcome matrix. Its classifier results and feature-attribution claims require their own provenance and are not evidence that a particular feature causes ranking reversals. This draft retains the source and prompt-semantics caveats without using those classifier accuracies as central portability evidence.

5.6 Operating-Region Calibration

A FIFO-only calibration pass establishes six operating regions – `LOW` (load factor 0.5), `PRE_KNEE` (0.8), `KNEE` (1.0), `POST_-KNEE` (1.1), `OVERLOAD` (1.2), and `HIGH_PRESSURE` (1.5) – across all 120 frozen windows (720 calibration cells total; 720 valid, deterministic, 0 failed/missing/ duplicated). We report this as calibration provenance, not a comparative scheduler result: FIFO is the calibration reference policy, and the calibration's role is to fix load-factor-to-region mappings before any other policy is executed against them, not to characterize FIFO's relative performance. The preserved 720-cell raw FIFO calibration manifest contains no zero-completion cells. Earlier manuscript prose reporting 301 such records is inconsistent with this artifact and is not retained as a finding. The distinction between calibration and campaign configuration is preserved: a calibration record is not an outcome for every policy under the final campaign.

Table 5. Range of window-level median source-reported prompts in the frozen 40-window/source corpus. These are not common-tokenizer counts.

Source	Minimum median	Maximum median	Medians above 256
azure_llm_2024	628.5	1595	40/40
bailian_qwen	343.5	1150	40/40
burstgpt	140.5	620.5	9/40

5.7 Paired Design and Repetitions

Because the simulator is fully deterministic (§4.2), a single execution per (window, policy, region) cell is sufficient: each of the 9,360 configurations yields one scientific outcome, and its rep0/rep1 pair (18,720 executions in total) is a deterministic verification pass confirming bit-identical output, not an independent stochastic replicate. Ranking-uncertainty quantification (§7) instead comes from resampling over the workload windows themselves (block bootstrap), not from repeated stochastic runs of the same cell.

5.8 Protocol freeze, pilot and overlap

The repository preserves the workload, panel, calibration, campaign and analysis freezes and their hashes. The earlier discriminability pilot motivated the larger panel; it is not confirmatory ranking evidence. The principal outcome matrix is fixed. Subsequent metric and SLO analyses are labeled extensions, with their original configurations unchanged. Reused pilot windows are disclosed rather than counted as an independent replication of the pilot. Historical source and freeze identities are retained even where later prose or engineering audits qualify their meaning.

A related load-scaling manuscript and a separate module-intervention project share research lineage. This paper does not claim their results or intervention-based attribution as its own. The earlier 60-window project's exact byte-level overlap is not fully reconstructed; final overlap review remains necessary. The present sampler is distinct, but that alone is not a proof of zero overlap.

5.9 Window and trace eligibility

Every principal window contains 200 requests; a sweep of requests per window has not been performed. The recovery analysis varies the number of windows, not requests per window. Source-reported token counts are not retokenized model inputs. All 40 Azure and 40 Bailian window medians exceed 256 source tokens. The later whole-prefill engineering contract, bounded at 256 prompt tokens and 16 active sequences, therefore does not cover these traces. No requests were truncated or filtered for this manuscript preparation, and no mapping to a modern model/tokenizer context limit is silently assumed.

5.10 Temporal partitions

The manifest assigns 20 EARLY, 10 MIDDLE and 10 LATE windows per source (60/30/30 over all three sources). Thus 60/30/30 is not a BurstGPT-only count. BurstGPT uses relative-chronology terciles and an additional bisect sensitivity split. Azure uses the frozen 2024-05-15 calendar boundary; Bailian is labeled relative chronology only. These definitions remain unchanged and are interpreted separately.

5.11 Recorded prompt-length coverage

These summaries demonstrate a coverage gap for bounded short-prompt serving checks, not an exact request-exclusion fraction. A window median does not identify every request above a threshold, and source length fields do not recover absent prompt text. No benchmark text is synthesized here to claim tokenizer equivalence. Model, tokenizer and context eligibility for prospective native traces require separate decisions and measurements.

6 Performance Within Operating Regions

The operating-region names identify FIFO-referenced arrival-rate settings, not independently measured physical saturation states. Each source has 40 windows and every primary policy is observed on each window in each region. Comparisons within a source/region use matched windows; a policy is not calibrated separately to obtain a favorable operating point.

The central quantity is relative order under ANWG, the fraction of arrival priority weight receiving deadline credit. Completion-conditioned latency and throughput can describe a different subset of requests. A policy that finishes only favorable requests can have a strong completion-conditioned metric while losing arrival-normalized utility; the common denominator and missing-value rules are therefore part of the comparison, not merely reporting details.

Across the complete two-pass campaign, 211 of 9,360 unique configurations have zero completions; 48 of these belong to the 11-policy primary panel. The preserved FIFO calibration manifest instead has zero zero-completion records out of 720. Calibration records cannot be propagated as zero-completion outcomes for all other policies. The reported rankings retain defined policy/window observations and omit undefined conditional observations, so effective observation sets can differ even when a condition-level ranking remains estimable.

The full per-policy outcome matrix remains the record for within-region performance. This paper does not turn the panel into a universal winner recommendation: the policies share a simulator implementation boundary, and region names, synthesized deadlines and metric conditioning all constrain the interpretation of an ordering.

6.1 Absolute utility before rank comparison

Table 6 is a new descriptive tabulation of the frozen outcomes, not a new execution or a replacement estimate. It reports the mean ANWG of FIFO, the range of policy means, and the size of the exact maximizing set. Each mean has the same 40-window support. Rounding in the table does not determine ties; sets are computed from unrounded values.

At LOW the maximum mean is 1.000 for every source, with six, seven and nine policies tied in Azure, Bailian and BurstGPT respectively. Yet the minimum is 0.018 for Azure, 0.080 for Bailian and 0.968 for BurstGPT. A ranking difference at LOW therefore has quite different absolute-utility context across sources. The table also explains why a tie-broken top choice is a weak description of a unique preferred policy in this region.

At HIGH_PRESSURE, FIFO means are 0.964, 0.946 and 0.645 respectively. The highest means remain 1.000 for Azure and Bailian, whereas BurstGPT's maximum is approximately 0.652. Thus similar ordinal agreement can coexist with different attained utility. A high cross-source tau does not establish similar service quality; a lower tau does not identify the magnitude or cost of every changed pair. This is why Q1's ordinal summary is accompanied by Q2's practical-margin criterion rather than interpreted alone.

These contrasts cannot be attributed solely to queue ordering. Classical and serving-inspired implementations have different historical service and resource semantics (Section 4.2). In particular, instant-prefill and prefill-aware paths do not impose the same work model. Policies without hard KV enforcement can also report demand above

Table 6. Within-source ANWG context from frozen outcomes. Minimum and maximum are over the 11 policy means, not confidence limits. Top-set size uses exact stored arithmetic. A=Azure, B=Bailian, G=BurstGPT.

Source	Region	FIFO	Minimum	Maximum	Top-set size
A	LOW	1.000	0.018	1.000	6
A	PRE_KNEE	1.000	0.018	1.000	5
A	KNEE	0.998	0.018	1.000	4
A	POST_KNEE	0.991	0.018	1.000	4
A	OVERLOAD	0.987	0.018	1.000	4
A	HIGH_PRESSURE	0.964	0.018	1.000	1
B	LOW	1.000	0.080	1.000	7
B	PRE_KNEE	1.000	0.080	1.000	6
B	KNEE	0.998	0.080	1.000	4
B	POST_KNEE	0.988	0.080	1.000	4
B	OVERLOAD	0.979	0.080	1.000	4
B	HIGH_PRESSURE	0.946	0.079	1.000	1
G	LOW	1.000	0.968	1.000	9
G	PRE_KNEE	1.000	0.856	1.000	7
G	KNEE	0.997	0.666	0.997	4
G	POST_KNEE	0.981	0.573	0.981	3
G	OVERLOAD	0.935	0.490	0.936	2
G	HIGH_PRESSURE	0.645	0.271	0.652	2

nominal capacity. The table supports description of the resulting abstractions; it does not show that one native serving mechanism dominates another. This is a fundamental comparability boundary for a scheduler-selection claim, not an uncertainty that more resampling can remove.

6.2 Observation support and operating-region interpretation

All 48 zero-completion primary configurations belong to the historical vLLM policy: seven Azure windows and one Bailian window in each of six regions. Other primary policies provide defined completion outcomes on those same inputs. Consequently, the corresponding conditional vLLM means have 33 Azure or 39 Bailian observations, while other policy means use 40. ANWG includes all 40 windows and gives the zero-completion outcomes zero credit. These are different questions about the same frozen executions.

The zero-completion pattern is not a finding that each of those windows is intrinsically infeasible for all policies. It is specific to an implementation and its model path. Similarly, FIFO-referenced load factors are controlled arrival-rate settings, not measured native saturation states shared by every implementation. Interpreting higher-region changes requires both the absolute outcomes and these implementation constraints. The corpus does not estimate a universal utilization knee or a calibrated capacity curve for production hardware.

7 Portability Across Workloads and Regions

7.1 Cross-Source Rankings

Across the 3 source pairs × 6 operating regions (18 conditions), Kendall's tau-b for the PRIMARY metric ranges from 0.55 to 1.00, with a per-region mean between 0.69 and 0.77 (Figure 1). Every one of the 18 conditions has a positive, non-trivial tau-b: comparative scheduler rankings are never uncorrelated or inverted *in aggregate* between any two

sources at any region. Portability is, however, visibly uneven across source *pairs* rather than uniform: Azure-2024 and Bailian/Qwen agree very strongly with each other at every region (τ = 0.89–1.00), while every comparison involving BurstGPT is substantially lower (τ = 0.55–0.71) – a gap of roughly 0.2–0.4 in Kendall's tau-b that holds at all six regions, not just one. Top-1 agreement (the deterministic single top-choice policy) and top-3 agreement follow the same source-pair pattern in the underlying evaluation data; we do not tabulate top-1/top-3 separately in the main text to avoid restating the same qualitative pattern three times, but the full table is part of the preserved artifact (§11).

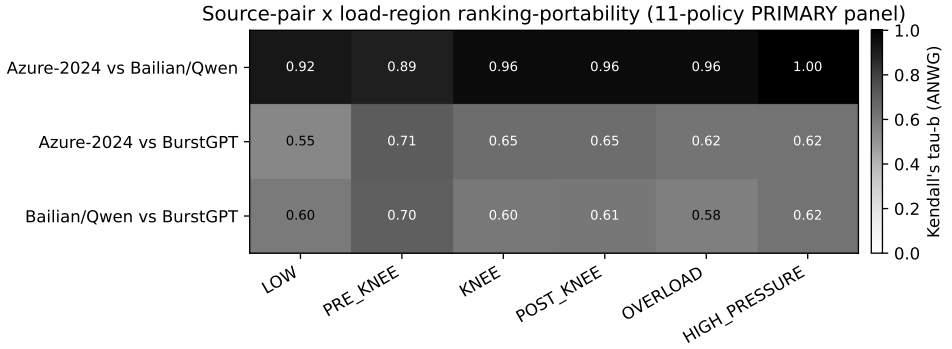


Fig. 1. Source-pair × load-region ranking-portability: Kendall's tau-b between each pair of sources' PRIMARY-panel (11-policy) rankings on the ANWG metric, at each of the six frozen operating regions (§5.6). Computed from the frozen ranking correlations artifact.

The min/max range in Figure 1 makes clear that portability is not homogeneous even within a region: the range at LOW (0.55–0.92) is nearly as wide as at any other region, and the range is driven entirely by which source pair is compared, not by which region is compared (§7.2 describes region dependence). No region shows uniformly low or uniformly high agreement across all three source pairs simultaneously.

7.2 Load Conditioning

Load-level dependence is a secondary robustness/sensitivity analysis (§3), reported here as a direct read of the same §7.1 table rather than a separate headline claim of cross-source rank instability by itself. Secondary ranking statistics (Spearman's ρ and top-1/top-3 overlap) provide additional summaries and are omitted from the main tables for concision; their full values are preserved in the preserved analysis outputs. Figure 1's per-region mean tau-b is close to flat across the six regions (0.69 at LOW, rising to 0.72–0.77 elsewhere, no monotonic trend with load factor); a region-subset comparison between the full six-region grid and the four-region `LOW/PRE_KNEE/KNEE/ OVERLOAD` subset used by `LOAD_-CALIBRATION_SENSITIVITY` (§9) changes the mean tau-b by less than 0.01 (0.728 on the four-region subset versus 0.733 on the full six-region grid) – this particular descriptive mean changes little under that region subset. Load regime does, however, materially affect *where practically supported reversals occur* (§7.3): reversal counts are concentrated at PRE_KNEE and above, essentially absent at LOW.

Table 7 separates small ordering changes from supported reversals. No primary record is unestimable or a practical sign change with a wide saved interval. This does not prove that every microscopic sign change is statistically insignificant: the frozen classification stops at the margin test and need not evaluate its interval. The absence of supported reversals at LOW is an observed result under the fixed metric and margin, not a reason to exclude that region.

Table 7. Primary ANWG cross-source comparison records by region. Each row contains 55 policy pairs times three source pairs; these are overlapping comparisons, not independent experimental units.

Region	Stable	Microscopic	Supported	Total
LOW	161	4	0	165
PRE_KNEE	149	10	6	165
KNEE	149	10	6	165
POST_KNEE	147	12	6	165
OVERLOAD	147	8	10	165
HIGH_PRESSURE	145	12	8	165

7.3 Pairwise Reversals

Of the 990 (policy pair) × (source pair, region) PRIMARY-metric records (55 policy pairs among the 11 PRIMARY policies × 18 conditions), the five-class breakdown (§3.3) is: 906 `STABLE_NO_SIGN_CHANGE` (91.5%), 48 `MICROSCOPIC_SIGN_CHANGE` (4.8%), 36 `SUPPORTED_PRACTICAL_REVERSAL` (3.6%), and zero records in either `UNDEFINED_UNESTIMABLE` or `UNSUPPORTED_SIGN_CHANGE_WIDE_CI`. The large majority of pairwise comparisons are stable; a small but non-trivial minority (about 1 in 28) are both practical reversals passing the saved nominal interval rule – never a majority, and never absent. The saved taxonomy contains no practical sign change with a wide interval. Microscopic changes are tested before the interval branch, so this is not evidence that practical and statistical criteria always agree.

Reversal counts are load-dependent: zero `SUPPORTED_PRACTICAL_REVERSAL` records at `LOW`, rising to 6 at each of `PRE_KNEE`, `KNEE`, and `POST_KNEE`, 10 at `OVERLOAD`, and 8 at `HIGH_PRESSURE` (Table 7). All 36 involve BurstGPT, but they span five policy pairs: SLAI–vLLM (10), Sarathi–SLAI (9), chunked-vLLM–SLAI (9), Sarathi–vLLM (4), and chunked-vLLM–vLLM (4), where these names abbreviate historical simulator identifiers. Earlier manuscript wording attributing all 36 records to one pair was incorrect. The single largest-effect-size supported reversal (operationalized as the smaller-magnitude of the two conditions' practical margins, the binding constraint under the frozen practical-margin rule, §3.3) is `slai_faithful` vs. `vllm_faithful` at `HIGH_PRESSURE`, comparing Azure-2024 (or, at an effectively tied effect size, Bailian/Qwen) against BurstGPT – this is the case selected for the historical physical comparison under the frozen selection rule (§10.2).

A representative stable comparison, used as a null-result control alongside the reversal case in the historical physical design (§10.2), is the source pair with both the highest tau-b and (among the highest-tau conditions) the narrowest bootstrap interval: Azure-2024 vs. Bailian/Qwen at `HIGH_PRESSURE` ($\tau = 1.00$, nominal 95% resampling interval $[0.90, 1.00]$).

7.4 Temporal Portability

Following the frozen temporal design (§5.10), four temporal record types are reported separately, each under its source's own chronology, and never merged into a single "temporal/domain OOD" number: BurstGPT's EARLY/MIDDLE/LATE tercile split (primary) and EARLY/LATE bisect (sensitivity check on the tercile boundary), Azure-2024's absolute calendar split, and Bailian/Qwen's `RELATIVE_CHRONOLOGY_ONLY` comparison, for which no calendar-based claim is made. All values are keyed identically to the source temporal robustness artifact. The temporal evidence is mostly stable, with a LOW exception: BurstGPT's tercile split shows strong agreement between EARLY and LATE chunks across non-LOW load regions ($\tau_b \in [0.81, 1.0]$), while the BISECT sensitivity check yields similarly stable correlations ($\tau_b \in [0.72, 0.80]$). At LOW, one BurstGPT tercile comparison instead has $\tau_b = -0.10$; this exception must not be hidden by a non-LOW

Table 8. All 18 primary source-pair comparisons. A=Azure, B=Bailian, G=BurstGPT. Intervals are preserved nominal 95% window-resampling intervals; M2 independently reproduced all 18, without establishing coverage.

Region	Pair	Tau-b	Saved nominal interval
LOW	A–B	0.920	[0.864, 0.920]
LOW	A–G	0.551	[0.203, 0.551]
LOW	B–G	0.599	[0.220, 0.599]
PRE_KNEE	A–B	0.893	[0.893, 1.000]
PRE_KNEE	A–G	0.709	[0.450, 0.753]
PRE_KNEE	B–G	0.697	[0.404, 0.753]
KNEE	A–B	0.958	[0.914, 1.000]
KNEE	A–G	0.646	[0.578, 0.729]
KNEE	B–G	0.604	[0.521, 0.688]
POST_KNEE	A–B	0.958	[0.958, 1.000]
POST_KNEE	A–G	0.647	[0.510, 0.678]
POST_KNEE	B–G	0.606	[0.469, 0.647]
OVERLOAD	A–B	0.958	[0.958, 1.000]
OVERLOAD	A–G	0.620	[0.566, 0.620]
OVERLOAD	B–G	0.580	[0.525, 0.620]
HIGH_PRESSURE	A–B	1.000	[0.904, 1.000]
HIGH_PRESSURE	A–G	0.623	[0.550, 0.712]
HIGH_PRESSURE	B–G	0.623	[0.550, 0.699]

summary. Azure's calendar split produces near-perfect correlation ($\tau_b \in [0.90, 1.0]$), and Bailian's relative-chronology split likewise exhibits minimal rank change ($\tau_b \in [0.94, 1.0]$).

Although the frozen protocol for this research question specified an analysis of temporal, provider, and domain shifts, a standalone domain-shift component was not separately operationalized in the final artifacts. The evidence reported here addresses the temporal component directly. The frozen cross-source comparisons (§7.1) include the one genuine provider boundary in this corpus (Bailian/Qwen on Alibaba Cloud versus the two Microsoft-infrastructure sources, §5.1), but the design does not isolate a causal provider effect from ordinary cross-source variation – BurstGPT-vs-Azure is a cross-source, not cross-provider, comparison; domain shift, like the provider component, is addressed only implicitly through these same cross-source comparisons and was not separately operationalized.

7.5 Point estimates and saved uncertainty

The intervals describe a window-resampling procedure on the sampled sources, not uncertainty over hardware architectures or all production populations. Their width is distinct from between-source variation in point estimates. Reporting them next to the full comparison set avoids selecting only the strongest or weakest agreement. Stable source pairs and the absence of a supported reversal at LOW remain part of the result.

8 Portability Across Metrics

Q3 asks whether a preferred-policy conclusion survives a change of objective. This section uses the saved post-campaign cross-metric extension and new, explicitly post-hoc M2 sensitivities on its frozen inputs. No serving execution is added. All 55 unordered pairs among the 11 eligible metrics are evaluated for each of three sources and six regions: 990 conditions. These conditions share outcomes and are not independent observations.

Table 9. Historical and post-hoc agreement summaries on the same 990 cross-metric conditions. Different definitions answer different questions.

Agreement definition	Count	Fraction
Historical deterministic ID	316	31.9%
Post-hoc nonempty maximizing-set intersection	572	57.8%
Post-hoc equality of maximizing sets	179	18.1%
Both objectives have unique maxima (eligibility)	257	26.0%

8.1 Historical correlations and classification counts

The historical median Kendall tau-b is 0.419, with range $[-0.506, 1.000]$; 202 of 990 conditions have negative correlation. The deterministic top choice agrees in 316 conditions (31.9%). Thus the objective can change both the full ordering and the selected policy ID. The median describes this enumerated set of metric pairs; it is not an expectation over a population of future deployment objectives.

The 54,450 policy-pair records divide into 45,800 same-order cases, 5,205 microscopic sign changes, 201 practical sign changes with a wide saved interval, and 3,244 saved-supported practical disagreements. The historical screened BH procedure flags 3,207 of the last group, removing 37. These are preserved procedural counts. M2 withdraws the stronger interpretation that they are statistically established practical effects at a controlled error rate. The limitations of the component tail quantities, selection and dependence apply here as in Q2.

No condition-level ranking is undefined, but that does not imply complete policy/window observations. Nor do 54,450 records supply 54,450 samples: each window contributes to multiple policies, metrics and regions. The number of pairwise records is useful for describing the evaluation grid, not for justifying precision of a claim about general serving workloads.

8.2 Tied optima versus deterministic policy selection

Let $T_m = \{p : \bar{y}_{p,m} = \max_j \bar{y}_{j,m}\}$ after direction normalization. The historical rule selects the smallest policy ID in T_m. A set-based question instead asks whether $T_a \cap T_b$ is nonempty: is there any policy maximizing both observed objectives? Equality of the two sets asks a stricter question. Neither set criterion selects a policy without an additional preference rule.

Among 1,980 metric sides, 1,010 have tied maxima. In the M2 post-hoc analysis, the maximizing sets intersect in 572/990 conditions (57.8%), and are identical in 179/990 (18.1%). Only 257 conditions have a unique maximizer on both sides. Mean Jaccard overlap, $|T_a \cap T_b|/|T_a \cup T_b|$, is 0.329. These statistics use exact aggregate equality, matching the historical arithmetic; no tolerance was tuned to improve agreement. Near-optimal sets under a deployment-specific utility tolerance are a different, unmeasured object.

The additional 256 intersecting-set conditions show why the complement of 31.9% cannot be called disagreement between unique optima. A common best policy may exist even when the ID rule picks different members of unequal sets. Conversely, nonempty intersection can be permissive when one objective ties many policies. Reporting intersection alongside set size and equality avoids presenting 57.8% as a corrected version of 31.9%. Both are valid descriptions of different selection questions.

8.3 Matched support and the meaning of a metric difference

Historical policy means omit undefined observations separately. To compare a and b, the difference of those means is

$$d_{\text{sep}} = \frac{\sum_{w \in \mathcal{W}_a} y_{w,a}}{|\mathcal{W}_a|} - \frac{\sum_{w \in \mathcal{W}_b} y_{w,b}}{|\mathcal{W}_b|}, \tag{5}$$

whereas the bootstrap difference averages $y_{w,a} - y_{w,b}$ over $\mathcal{W}_a \cap \mathcal{W}_b$. Without equal support, its interval is not an interval for d_{sep}. This is an estimand mismatch, not simply a wider or narrower error bar. M2 therefore avoids assigning that interval to the separately aggregated point difference.

We perform two distinct post-hoc checks. First, each metric-pair ranking is recomputed over windows where all 11 policies have both metrics. This common-panel support retains 33–40 windows per condition. The median tau becomes 0.41849, 202 correlations remain negative, and the ID agreement remains 316/990. Individual tau values change in 81 conditions. The historical median remains reported as 0.419; the new value is a sensitivity estimate with a different eligibility rule.

Second, for every policy-pair record, each metric's policy means are recomputed over that pair's observed-window intersection. This retains more observations than requiring the entire panel. Differences change in 5,890 of 54,450 records, including 2,720 of the 3,244 saved-supported records. All 3,244 retain opposite signs and both 10% point-margin gates; for the affected saved-supported records, the matched signs also agree with the saved interval signs. This supports robustness of those *descriptive gates* in this corpus. It neither validates their p-values nor supplies fresh intervals for newly eligible cases. We do not report a replacement total of statistically supported disagreements.

Matching support also changes the scientific target. A latency comparison restricted to windows on which both policies finish requests does not represent windows where one finishes none. Within an observed window, each policy's latency may still describe a different set of completed requests. Matched windows therefore do not eliminate request-level survivorship selection. ANWG answers an arrival-based question; conditional latency answers a completion-based question. Neither should be silently substituted for the other.

8.4 Implications for objective-dependent selection

A policy can have low latency among its completed requests while providing little arrival-normalized deadline utility. Throughput and TTFT measure other aspects of service; priority weighting changes whose outcomes count most. Some disagreement between such rankings is consequently compatible with internally consistent measurements. Negative tau does not by itself identify a measurement error or a universally inferior scheduler.

For evaluation practice, the objective, denominator and eligibility rule must accompany the selected policy. If the intended decision is to attain arrival deadlines, completion-conditioned speed is insufficient evidence. If several policies attain the same measured objective, the benchmark can report that set rather than implying that an arbitrary ID is uniquely best. A deployment may use another objective to break ties, but this corpus does not validate that decision rule, its cost or its portability.

The saved SLO extension in Section 9.3 changes the benchmark's synthetic deadline convention rather than merely relabeling an unchanged latency statistic. Its 13/30 top-choice changes and six reversal-direction changes among 72 nonidentity checks show dependence on that convention within its selected scope. The extension excludes LOW because its design targeted saved reversal cases; no conclusion about LOW's deadline sensitivity follows. These distinctions answer Q3 without turning retrospective sensitivity into a prospective deployment guarantee.

9 Robustness and Practical Implications

9.1 Robustness

Of the seven non-SLO robustness items in the historical implementation account (§9), six were computed (five in the ranking-correlation, temporal-robustness, and sample-complexity outputs below, and a sixth, `LEAVE_ONE_POLICY_FAMILY_OUT`, in the telemetry-explanation analysis of §9.4). The seventh, `METRIC_DEFINITION_SENSITIVITY`, was never implemented; we disclose this as a protocol gap rather than perform a post hoc analysis after the results were already known.

- **`PRIMARY_ONLY`.** Identical to the headline result by construction: the ranking correlations artifact already restricts every comparison to the 11-policy PRIMARY panel (§7.1), never mixing in the two `STYLE_APPROXIMATION` robustness policies.
- **`LEAVE_ONE_SOURCE_OUT`.** Excluding BurstGPT leaves only the Azure-2024/Bailian-Qwen pair, whose mean tau-b is 0.948 – close to the ceiling. Excluding Azure-2024 or Bailian/Qwen each leaves a BurstGPT-involving pair, with mean tau-b 0.618 and 0.633 respectively. This is the same asymmetry as §7.1, now stated per excluded source: the headline portability figure is high specifically because two of the three sources agree strongly, and is pulled down specifically by BurstGPT, not uniformly by all three sources.
- **`LOAD_CALIBRATION_SENSITIVITY`.** Reported in §7.2: mean tau-b changes by less than 0.01 between the four- and six-region grids.
- **`TEMPORAL_SPLIT_SENSITIVITY`.** The BurstGPT bisect-vs-tercile comparison from §7.4; both split definitions are reported side by side in the preserved table rather than collapsed into one number.
- **`WINDOW_SIZE_SENSITIVITY`.** Despite its name, this item is the window-*count* subsampling ladder of §9.2, not a requests-per-window sweep (§5.9), and is not separately recomputed here.
- **`LEAVE_ONE_POLICY_FAMILY_OUT`.** Excluding any single one of the ten frozen mechanism families changes the headline mean tau-b only modestly, from 0.707 (excluding the classical arrival-order family) to 0.792 (excluding the SLO-aware family), against the full-panel baseline of 0.733. No single mechanism family's removal collapses or inflates the headline portability figure – the source-pair-driven pattern of §7.1 is not an artifact of any one policy family dominating the panel.

9.2 Finite-Corpus Ranking Recovery

Scope. The reference is the frozen 40-window ranking, not an unknown population optimum. Recovery is conditional on this finite corpus and the historical tie rule.

Design and results. The 0.9 recovery probability used throughout this section is a pre-registered benchmark reporting target, fixed before any sample-complexity output was computed: it provides one interpretable, high-recovery criterion for comparing sources and metrics against each other, and is not asserted to be a universal statistical-significance level or a field-standard adequacy cutoff for scheduler benchmarking.

For the PRIMARY metric, recovery of the exact $n = 40$ reference ranking is slow and strongly source-dependent (Figure 2): BurstGPT never reaches the preregistered 0.9 exact-order recovery threshold below the full window budget (0.2% at $n = 5$, rising only to 56.6% at $n = 30$), Bailian/Qwen similarly does not cross 0.9 before the $n = 40$ ceiling (64.0% at $n = 30$), while Azure-2024 crosses the threshold already at $n = 30$ (98.8%). None of the three sources reaches 0.9 exact-order recovery at $n \in \{5, 10, 20\}$. Because $n = 40$ is the size of the full reference pool itself, exact recovery at

$n = 40$ is 1.0 by construction (sampling all 40 windows without replacement reproduces the reference exactly) and is not an informative data point on its own; the informative content is in how slowly each source's curve rises toward that trivial ceiling.

Top-1 recovery (does the resampled subset reproduce the deterministic PRIMARY-panel top choice) is a materially easier question than exact-order recovery for two of the three sources: Azure-2024 and Bailian/Qwen both exceed 0.9 top-1 recovery already at $n = 5$ (96.6% and essentially 100% respectively), even though their *exact*-order recovery at $n = 5$ is only 35.0% and 14.6%. For these two sources, recovering that top choice requires fewer sampled windows than reconstructing the complete 11-policy ordering. BurstGPT shows no such gap: its top-1 recovery is comparably slow to its exact-order recovery (13.0% at $n = 5$, 74.2% at $n = 30$), remaining below 0.9 until the trivial $n = 40$ ceiling – consistent with §7.1's finding that BurstGPT is the source whose relative ordering is hardest to pin down. Comparing a concentrated (single-source-heavy) against a distributed (evenly-spread-across-sources) window budget of equal total size, the distributed budget achieves higher mean Kendall's tau-b against the reference ranking (0.986) than the concentrated budget (0.940) for the primary metric – spreading a fixed evaluation budget across sources recovers the reference ranking more reliably than concentrating it in one source.

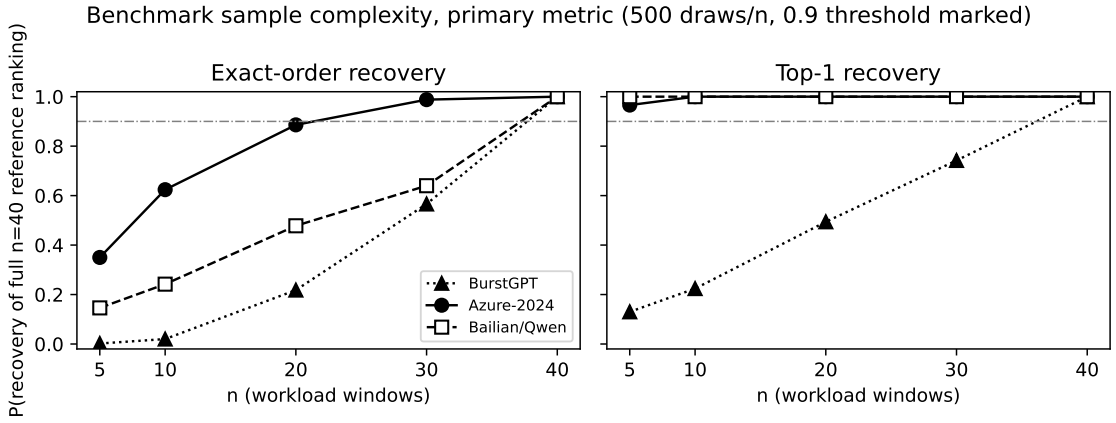


Fig. 2. Benchmark sample-complexity recovery curves, PRIMARY metric, 500 draws per n, sampling without replacement. Left: exact-order recovery of the full $n = 40$ reference ranking. Right: top-1 recovery. The 0.9 threshold is marked. Computed from the frozen sample complexity artifact.

This analysis partially addresses original RQ5 and answers the finite-reference part of Q4 (§3) and depends on the full primary outcome matrix (§7); it is not run on the earlier, smaller discriminability pilot (§5.8).

9.3 SLO-Definition Sensitivity

As disclosed in §9.4, the concrete extension was specified after primary results, although broad SLO sensitivity was already anticipated in the historical plan; we report it here as the post-campaign extension it was actually run as: three alternative SLO-synthesis multipliers (`tight_10x`, `primary_20x` – this study's own convention – and `loose_40x`) applied across the 5 load regions that contain at least one PRIMARY-metric supported reversal (excluding `LOW`, which accounts for 0 of the 36 supported reversals and so the selected-reversal persistence design omits it; this does not establish SLO insensitivity at LOW), evaluating 11 policies × 120 total windows × 5 regions × 3 variants (19,800 input rows according to the preserved extension status), with every seed held identical to §7.

Ranking robustness. Across the 30 (source, region, variant) conditions compared against the study's primary SLO convention, rank correlation stays high under either alternative (median Kendall's $\tau_b = 0.872$, range $[0.533, 1.000]$); the single top-ranked policy nonetheless changes under an alternative SLO definition in 13/30 (43%) of conditions.

Reversal persistence. We check the 36 PRIMARY-metric supported reversals from §7.3 against the alternative definitions. Because there are 3 SLO variants (tight, primary, loose), there are $36 \times 3 = 108$ mathematical comparisons in the artifact's persistence matrix, of which 36 are identity checks against the primary definition itself (all 36 trivially persist). Restricting focus to the 72 scientifically meaningful non-trivial comparisons against the alternative definitions: 52/72 (72%) persist, 6/72 (8%) change direction, 10/72 (14%) remain the same direction but fall below the practical-margin threshold under the alternative definition, and 4/72 (6%) become unsupported by the extension's saved interval rule. The six direction changes contradict a claim of invariant reversal direction under alternative deadlines. A non-trivial minority (20/72, 28%) are sensitive to the SLO-definition choice – disclosed here as a caveat on those specific reversals, not grounds to discard them.

9.4 Explanations and unresolved questions

The saved descriptor regression has 42 usable fits among 990 records; no standard errors establish a directional explanatory claim. Source identity, prompt structure and simulator execution semantics remain confounded. The descriptive association is not a causal mechanism result or a deployment selector. The synthetic-to-real pilot is engineering context only; it does not answer RQ3 at the level of the primary corpus.

The SLO extension was specified after the primary outcomes were known. The alternative-metric-definition robustness item remains distinct from that deadline-synthesis extension. Neither replaces a missing prospective analysis, and requests-per-window sensitivity is still unmeasured.

Practically, a ranking should travel with its implementation identity, workload/window definition, calibration, metric denominator, tie rule and uncertainty procedure. Agreement on that record is evidence for the specific comparison. A changed objective or execution model warrants reassessment; it does not license assuming a reversal. No held-out portability predictor or cost-effective recalibration rule is validated here.

9.5 Independent inferential audit and bounded sensitivities

The M2 audit independently reconstructs all 198 saved cross-source tau intervals across 11 metrics, including the 18 primary intervals, and all 72 intervals underlying the 36 primary saved-supported reversals. It also reconstructs 6,890 cross-metric difference intervals and six rank intervals covering ANWG versus mean latency at LOW and HIGH_-PRESSURE in all sources. The independent script imports no production analysis routines. All checked endpoints and tail fractions match to numerical tolerance.

Reproduction requires more than a nominal seed. The primary pipeline uses one NumPy random stream initialized at zero, consumed first by all 198 ranking comparisons and then by reversal comparisons reaching the interval stage. Its row order and preceding draws matter. The extension derives separate CRC32 seeds from condition, metric and policy keys. M2 records input hashes, ordering, seeds, algorithms and software versions. These facts establish computational repeatability, not empirical coverage.

The 66 Friedman records. For each metric/region, the blocks are all 120 windows, pooled across sources, and treatments are policies. The complete-policy rule includes all 11 policies for the three always-defined metrics (18 records), but excludes the historical vLLM policy for each conditional metric (48 records), leaving 10 treatments. It does not drop

those policies only within the incomplete source. Independent average-rank sums with tie correction reproduce every statistic and its chi-square tail probability; the audit supplies each exact input matrix.

This omnibus tests a policy main effect in the pooled block design under its null assumptions. It is not a source-by-policy interaction test and cannot establish cross-source ranking heterogeneity. The pipeline computes it first but does not condition pairwise analysis on its rejection. The pairwise reversal claim thus receives no additional validation from a small omnibus p-value. Independence of blocks and the suitability of the asymptotic null approximation remain assumptions, even with 120 nominal blocks. Likewise, saved Kendall/Spearman analytic p-values concern association across the selected policy coordinates, not uncertainty from sampling workload windows; we do not use them as window-level evidence.

Multiplicity and finite resolution. Thirty of 36 primary intersection–union tail quantities are zero in 2,000 draws. A zero means no relevant tail draw occurred, not zero error probability. As an arithmetic sensitivity, M2 retains the complete family size, assigning untested records a placeholder of one. This is not a new calibrated test. The number of saved-supported records also passing BH changes from 36 to 30 for the primary analysis, and from 3,207 to 3,006 for the cross-metric extension.

Adding one to each tail count and dividing by 2,001 before doubling gives minimum two-sided value 2/2001. With full families, the corresponding counts are 30 and 2,971. Applying the harmonic-factor dependence adjustment of Benjamini–Yekutieli to those same quantities gives zero and 1,940. These different counts expose sensitivity to the testing interpretation; none is endorsed as an FDR-controlled discovery count. Valid null p-values are required even for a dependence adjustment. In particular, a Monte Carlo continuity adjustment cannot turn the uncentered empirical bootstrap into a proven null test.

Dependence stress test. A separate post-hoc calculation orders the 40 selected windows of each source by valid-row position and groups consecutive sets of five. It resamples eight groups with replacement for 2,000 draws, preserving policy pairing within each group. Thirty of the 36 primary saved-supported records retain zero-excluding intervals on both sides. This is a bounded stress test, not an estimated optimal block length or validated 95% procedure: selected windows are irregularly separated, and eight groups provide limited information. No provider-level bootstrap is justified by three sources spanning only two infrastructure lineages.

Together these checks support retaining the observed ordering changes and historical procedural labels while withdrawing their claimed error-rate interpretation. They do not show that the six sensitivity losses are false reversals, or that the remaining 30 are certain. More resampling of the same windows cannot create independent workload coverage, rectify policy comparability, or settle faithful native transfer.

10 Threats to Validity

10.1 Simulator validity and policy comparability

The primary findings are conditional on a project-local, deterministic, fixed-step simulator. Native partial-prefill execution, allocation refusal, KV preemption and recomputation are not reproduced merely by a policy's name or unit-test success. Historical policies can maintain shadow progress and KV tables that differ from shared execution state. Some resource limits are policy-dependent; demand above nominal KV capacity is possible. The simulator is therefore a comparison of these implemented abstractions, not a controlled physical comparison isolating only production scheduler selection rules. Differences in abstraction can influence rankings.

Later source audits reproduced discrepancies in final-prefill sampling, output-prefix retention under recomputation and admission accounting. Native scheduling counters are not execution acknowledgments. Intermediate prefill can produce no sample, while final prefill can produce the first output token. Restarting generated progress changes work and first-token accounting. No correction factor translates historical simulated timing to physical timing; no numerical ranking effect was measured by those audits.

10.2 Physical evidence boundary

The legacy RQ6 artifact records 240 physical executions of two configured policy paths across three sources and 40 windows per source. A retrospective hardware audit established 10 GB A100 MIG allocations, with 9.5 GiB visible to CUDA, rather than full 80 GB devices. Parent-card names were not allocated-device identity evidence. Individual allocated UUIDs and exact compute-slice profiles were not preserved.

More seriously, request class metadata was not transported to the legacy SLAI plugin; missing classes selected fallback behavior. Its deadline clock used a synthetic iteration-to-time conversion. Client worker limits and prompt construction could delay actual arrivals, which were not fully instrumented, and adapter plans could differ from native scheduled work. The saved numerical observations remain historical evidence of that runner. They do not establish faithful SLAI-versus-native ranking agreement or reversal non-reproduction. The mismatch cannot be attributed uniquely to simulator error. RQ6 remains unresolved at the intended fidelity level.

A later full-A100 engineering smoke established bounded native/checked-SLAI serving on short synthetic fixtures. It is separate from this campaign and does not validate realistic-trace ranking portability. Subsequent CPU ledger and native-boundary conformance work uses synthetic execution replies; it is not GPU timing, numerical-output or performance evidence. The P2B control hook rejects selected waiting work preceding selected resident work in one batch and has no atomic native-state rollback on late allocation failure. General partial-prefill serving, KV pressure/preemption, recomputation and distributed TP2/DP2 remain separate gates. Nothing in this manuscript establishes cross-architecture or parallelism portability.

10.3 Statistical and construct validity

The 18,720 executions comprise 9,360 deterministic pairs, not stochastic replications. Window bootstrap intervals summarize resampling variation within the sampled trace corpus. Their nominal coverage requires assumptions not established here. Production traces can retain temporal or session dependence across windows. Independent publication of traces is not independent sampling of providers; BurstGPT and Azure share infrastructure lineage. Provider and domain effects are not causally isolated. The temporal definitions differ by source and should not be pooled into a common domain-shift statistic.

Missing completion-conditioned observations are excluded, not imputed. An estimable aggregate does not imply that every policy/window observation exists. In the primary panel, 48 unique configurations have zero completions; the preserved calibration manifest has none. Earlier prose reporting 301 calibration zero-completion cells is disputed by that file. Survivorship conditioning, deterministic tie breaks and synthetic deadlines affect the meaning of a preferred policy. Practical-margin and recovery thresholds are reporting conventions, not universal systems standards. The raw-interval reversal taxonomy and within-family BH flags are historical procedural labels. Neither a calibrated significance level nor even within-family FDR control has been established. Increasing bootstrap draws cannot repair dependence, screening or a mismatch of estimands.

The cross-metric and SLO extensions were conducted after primary results were available. Their frozen extension configurations support reproducibility, not retrospective preregistration. The descriptor regressions are sparse and descriptive. The finite-reference recovery curves do not prove that 40 windows suffice for a new source or that a full-reference ordering is a population truth. Alternative requests-per-window sizes, held-out sources and a prospective transfer predictor are not evaluated.

10.4 Coverage, provenance and reproducibility

Source token counts may represent different context semantics and are not necessarily counts under a common tokenizer. Long prompts in the Azure and Bailian windows are outside the later 256-token engineering contract. No retokenization or context-eligibility audit can be inferred from the short smoke. The 120 selected windows are a finite corpus, not a representative sample of every production LLM service.

Historical source, campaign and analysis files are preserved with hashes. M1 reconciled counts and point estimates; M2 independently reproduces all 198 source-ranking tau intervals, 72 primary reversal intervals, 6,890 cross-metric difference intervals, six cross-metric rank intervals and 66 Friedman statistics. This does not establish interval coverage, fresh hardware reproducibility, source licensing or public release accessibility. Bibliography metadata and the prior manuscript overlap boundary require final author review. Unresolved checks are listed explicitly in the accompanying claim-to-evidence register.

11 Conclusion

The four retrospective manuscript questions have conditional, evidence-based answers. For Q1, aggregate ANWG orderings are positively associated across all 18 source-pair/region comparisons ($\tau_b = 0.551$–1.000), with stronger Azure–Bailian agreement than BurstGPT comparisons. Absolute utility and tied top sets differ markedly across sources, so ordinal agreement alone does not establish equivalent service quality.

For Q2, 36 of 990 records pass the historical 10% point-margin and nominal interval rule, spanning five policy pairs. The endpoints reproduce, but calibrated significance and FDR control are not established; alternative family bookkeeping and grouped-window resampling each retain 30 under their respective rules. The finding is a minority of observed practical ordering changes in a specified simulator, not 36 confirmed physical reversals.

For Q3, the historical median cross-metric tau is 0.419. Deterministic ID agreement is 31.9%, whereas maximizing sets intersect in 57.8% of conditions. Matching observed windows leaves the broad descriptive pattern intact but changes some estimates and their target population. Saved SLO sensitivities likewise constrain any objective-independent selection claim.

For Q4, most temporal comparisons are stable, with a BurstGPT LOW exception at $\tau_b = -0.10$. Recovery depends on source and target: the 0.9 exact-order criterion is reached at 30 windows for Azure and only at the trivial 40-window reference for the other sources. This is finite-reference stability, not an independent-window requirement for future workloads.

The contribution is a traceable comparison design and a critical empirical case study, not a new rank statistic or a universal scheduler recommendation. Original synthetic-to-real and faithful native-ranking questions remain unanswered. Policy-dependent service semantics and legacy physical-runner defects prevent transporting these results to production serving without separate comparability validation and new, prospectively designed evidence.

Artifact availability and declarations

The review repository preserves the source, configurations, frozen outcome matrix, canonical analyses and the M1 and M2 claim audits. The dataset package `dataset/v1.1.0` includes per-execution outcomes, analysis JSON and reproducible tabular views; it does not include a full dump of the legacy physical request records. Historical Hugging Face and Zenodo release references require a current accessibility and version check before submission. The working repository is private; this draft makes no claim that its new review files are publicly available. Upstream trace redistribution remains subject to the original terms.

The author declares no competing interests. Historical acknowledgments identify support from Google Cloud Research Credits, the CloudRift AI Builder Grant and the Cohere Labs Catalyst Grant Program, and acknowledge Professor Ioannis Koutis and NJIT's Wulver/ARCS resources. Funding and contribution statements require author confirmation for the final submission. The author is responsible for conceptualization, methodology, software, validation, analysis, data curation, visualization and writing. AI-assisted preparation was used historically and Codex assisted these M1/M2 revisions; the revised claims and text remain pending detailed author review and approval.

References

[1] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. VIDUR: A Large-Scale Simulation Framework for LLM Inference. In *Proceedings of Machine Learning and Systems (MLSys '24)*. MLSys. https://arxiv.org/abs/2405.05465

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association. https://www.usenix.org/conference/osdi24/presentation/agrawal

[3] Alibaba Cloud. 2026. *qwen-bailian-usagetraces-anon*. https://github.com/alibaba-edu/qwen-bailian-usagetraces-anon/tree/5f7439c51ec248a0c585f7d90a41a6f57773b912 Anonymized Bailian/Qwen production LLM-serving usage-trace release; repository `alibaba-edu/qwen-bailian-usagetraces-anon`, pinned commit `5f7439c51ec248a0c585f7d90a41a6f57773b912`, Apache-2.0.

[4] Agrim Bari, Parikshit Hegde, and Gustavo de Veciana. 2025. Optimal Scheduling Algorithms for LLM Inference: Theory and Practice. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 3 (2025), 59. doi:10.1145/3771574

[5] Yoav Benjamini and Daniel Yekutieli. 2001. The Control of the False Discovery Rate in Multiple Testing under Dependency. *The Annals of Statistics* 29, 4 (2001), 1165–1188. doi:10.1214/aos/1013699998

[6] Jaehong Cho, Hyunmin Choi, and Jongse Park. 2025. LLMServingSim2.0: A Unified Simulator for Heterogeneous Hardware and Serving Techniques in LLM Infrastructure. *IEEE Computer Architecture Letters* 24, 2 (2025). https://arxiv.org/abs/2511.07229

[7] Jaehong Cho, Minsu Kim, Hyunmin Choi, Guseul Heo, and Jongse Park. 2024. LLMServingSim: A HW/SW Co-Simulation Infrastructure for LLM Inference Serving at Scale. In *Proceedings of the 2024 IEEE International Symposium on Workload Characterization (IISWC '24)*. IEEE. https://arxiv.org/abs/2408.05499

[8] Janez Demšar. 2006. Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research* 7 (2006), 1–30. https://jmlr.org/papers/v7/demsar06a.html

[9] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS '24)*. https://arxiv.org/abs/2408.15792

[10] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S^3: Increasing GPU Utilization during Generative Inference for Higher Throughput. In *Advances in Neural Information Processing Systems 36 (NeurIPS '23)*. doi:10.5555/3666122.3666913

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM. doi:10.1145/3600006.3613165

[12] Microsoft Azure. 2024. *AzureLLMInferenceDataset2024*. https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2024.md

[13] William Nixon, Jon Durbin, Florian Standhartinger, Haryadi S. Gunawi, and Juncheng Yang. 2026. A Year in LLM Serving: Workload Evolution, Caching and Load-Balancing. *arXiv preprint* (2026). https://arxiv.org/abs/2608.13573

[14] Konstantinos Papaioannou and Thaleia Dimitra Doudali. 2024. The Importance of Workload Choice in Evaluating LLM Inference Systems. In *Proceedings of the 4th Workshop on Machine Learning and Systems (EuroMLSys '24)*. ACM, 39–46. doi:10.1145/3642970.3655823

[15] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA '24)*. IEEE/ACM.

doi:10.1109/ISCA59077.2024.00019

[16] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST '25)*. USENIX Association. https://arxiv.org/abs/2407.00079

[17] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association. https://arxiv.org/abs/2401.00588

[18] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. In *Proceedings of the 2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA '25)*. IEEE. https://arxiv.org/abs/2408.00741

[19] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association. https://arxiv.org/abs/2406.03243

[20] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider. *Proceedings of the 2025 USENIX Annual Technical Conference (ATC '25)* (2025). https://arxiv.org/abs/2506.02634

[21] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25), Volume 2*. ACM. doi:10.1145/3711896.3737413

[22] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. 2026. ServeGen: Workload Characterization and Generation of Large Language Model Serving in Production. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association. https://www.usenix.org/conference/nsdi26/presentation/xiang-servegen

[23] Mert Yildiz, Alexey Rolich, and Andrea Baiocchi. 2026. Dispatching policies in data center clusters: Insights from Google and Alibaba workloads. *Performance Evaluation* 172 (2026), 102551. doi:10.1016/j.peva.2026.102551

[24] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association. https://www.usenix.org/conference/osdi22/presentation/yu

[25] Netanel Zakay and Dror G. Feitelson. 2013. Workload Resampling for Performance Evaluation of Parallel Job Schedulers. In *Proceedings of the 4th ACM/SPEC International Conference on Performance Engineering*. ACM, 149–159. https://research.spec.org/icpe_proceedings/2013/proceedings/p149.pdf

[26] Netanel Zakay and Dror G. Feitelson. 2014. Workload resampling for performance evaluation of parallel job schedulers. *Concurrency and Computation: Practice and Experience* 26, 12 (2014), 2079–2105. doi:10.1002/cpe.3240

[27] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association. https://arxiv.org/abs/2401.09670

[28] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2026. TraceLab: Characterizing Coding Agent Workloads for LLM Serving. *arXiv preprint* (2026). https://arxiv.org/abs/2606.30560